

Toward a Neurosymbolic Software Factory: Bounded Proposal and Checked Transformation in ZenoFCIS

Dana Edwards

Draft—October 4, 2026

Abstract

ZenoFCIS is being built as a neurosymbolic software factory for high assurance functional-core/imperative-shell applications. Neural systems assist discovery and authoring; symbolic checks admit supported artifacts under declared contracts. This paper specifies a bounded optimization loop in which a model proposes a program or search strategy, optional equality saturation produces a canonical artifact, and an independent checker compares that artifact with an immutable original over a complete finite domain. Only a fully checked candidate satisfying explicit cost guards can replace the incumbent. We give conditional preservation and termination arguments, specify counterexample feedback, source-bound receipts, accounting and restart, and distinguish functional equivalence from application decisions, resources and authority. The empirical basis is an earlier symbolic-only study of 100 synthetic Boolean programs: semantic e-graphs reduced aggregate emitted instructions from 895 under local simplification to 816, with 18 wins, 78 ties and four regressions. That study’s protected node-count selection totaled 812. Two earlier extraction configurations failed; the successful configuration was exploratory. A current Python replay checks stored artifacts and results, without rerunning the Rust optimizer or Lean proofs. The integrated neural loop, pair-cost policy and target/application qualification have not been implemented or evaluated here. The contribution is a concrete assurance contract and evaluation plan, not a claim to invent neural proposal, counterexample-guided synthesis or translation validation, and not evidence of a complete or leading software factory.

1. A software factory with an explicit acceptance boundary

A software factory coordinates the path from requirements and source artifacts to checked software, integration, evidence and maintenance. Calling that factory neurosymbolic should identify a division of responsibility, rather than imply that a model’s confidence is a proof. In the proposed ZenoFCIS architecture, neural systems search for useful constructions, explanations and transformations. Symbolic mechanisms evaluate contracts, compare behavior and establish limited claims. An imperative shell coordinates models, files, tools and deployments; the functional core receives immutable data and returns owned results under library-defined rules. The intended product is a state of the art high assurance factory. That is an engineering objective, not a demonstrated comparative ranking.

The intended foundation is a **formally verified functional core**: pure, total decision functions proved against an explicit supported specification under named assumptions. V2 qualification is still incomplete. A proof about covered core bodies does not by itself prove the whole application or its imperative shell; those boundaries require their own integration and evidence gates.

This paper addresses one small part of that factory: replacing an admitted finite program with a cheaper equivalent program. It does not solve the adequacy of natural-language requirements,

general application synthesis, or deployment authorization. A completely correct optimizer can preserve a bug already present in the original program. A finite proof can also say nothing about inputs outside its declared domain. These are reasons to freeze and identify the contract, not reasons to enlarge the meaning of an equivalence receipt.

The central question is whether an arbitrary adaptive proposer can be useful without becoming a correctness authority. The answer is conditional: if a qualified checker establishes the required relation for the actual emitted artifact, the proposer may be unreliable without invalidating accepted results. Reliability then affects how often useful results are found, at what cost, rather than what a checked result means. The implementation must preserve that separation on every failure path, including parser failure, solver timeout, receipt replay, process death and session resume.

The proposed contribution has three parts: a frozen request and incumbent invariant; an implementable resource, feedback and integration contract; and a future evaluation that can falsify claims of neural benefit. The earlier Boolean study provides bounded feasibility evidence for one symbolic proposer [7]. It does not provide evidence for the new adaptive loop. Smaller ZenoFCIS V2 delivery remains first; this design extends the later 2.1 transformation track and adds no V2 completion condition.

2. Prior work and the scope of novelty

Equality saturation retains many equivalent expressions before extraction, avoiding some early rewrite-order commitments [1]. The egg library supplies an extensible implementation of that approach [2]. Semantic e-graphs can add equivalences derived from semantic information; recent EDA work studies this direction in a distinct setting [6]. Exact finite truth signatures, shared graphs and extraction objectives are established ideas. Our earlier experiment combines them for a small ZenoFCIS Boolean language rather than claiming a new general equality-saturation method.

Counterexample-guided inductive synthesis alternates candidate construction and verification [3]. The proposed loop is CEGIS-inspired, but a heuristic model need not satisfy accumulated constraints and may repeat the same failure. Therefore classical completeness arguments requiring a suitable synthesizer do not transfer merely because counterexamples appear in a prompt. Feedback changes the proposal context; this design does not train model weights.

LPO generates candidate peephole optimizations with language models, checks them using Alive2 and feeds diagnostics back into bounded retries [4]. EggMind studies language-model synthesis of equality-saturation strategies in a dedicated language [5]. Those works precede the broad proposal/check/feedback pattern used here. Their domains and results do not establish a ZenoFCIS gain. The possible contribution of this work is its particular application assurance contract, precise evidence boundaries and subsequent empirical evaluation. Acceptance follows a translation-validation style: validate each result rather than trust the discovery procedure. No novelty or optimality claim follows from adopting that architecture.

3. Programs, observations and frozen requests

Let P_0 be the original admitted program. A request R fixes P_0 's exact canonical bytes, the complete ordered input domain D , input/output ABI, execution semantics, observation and error profiles, codec, enumeration version, qualified checker identity, cost objective and resource/disclosure policy. The request retains these data, not only caller-supplied hashes. A domain-separated RequestId commits to the canonical request; its binding property assumes collision resistance. Exact admission

and custody remain necessary even when hashes are used for indexing.

The first profile, `FunctionalBoolV1`, admits zero to six Boolean inputs, one to sixteen ordered Boolean outputs and one to 256 eager acyclic instructions. The instruction set is `Input`, `Bool`, `And`, `Not` and `Select`. Every stored instruction is validated, including unused instructions and unselected arms. An OR operator used by a proposer must be lowered before acceptance. A zero-input program has one valuation, the empty tuple; it does not have an empty domain. Duplicate output positions, unused inputs and ABI ordering remain part of the contract. Domain shrinking, schema changes and unsupported instructions are refused.

For an admitted program P and input x , write $O_R(P,x)$ for its complete ordered observation. The desired relation is:

```
P equivalent_R Q  iff  same_schema_R(P,Q)
                        and for every x in D, O_R(P,x) = O_R(Q,x).
```

The Boolean profile is total after admission. An unexpected evaluator error prevents a successful equivalence receipt even if both sides fail identically. A later checked-integer profile may treat specified arithmetic errors as observations, but checker exhaustion is always separate from program failure. This distinction prevents incomplete work from masquerading as equality.

Canonical decoding must be bounded, consume all input, reject unknown fields and tags, and satisfy re-encoding equality. The checker evaluates the program decoded from the actual supplied bytes. A receipt for that DAG is not evidence about a separately printed or compiled Rust or Wasm artifact. Such a target requires its own correspondence contract.

Enumeration also needs executable qualification. The intended production product iterator advances the last ABI field fastest; the historical Boolean signature convention maps variable i to row-index bit i . Both can enumerate the same set, but a truth vector cannot be silently reinterpreted across the two orders. The checker needs a versioned ordinal-to-tuple bijection, complete termination and count checks. An expected count alone permits a duplicated row and an omitted input. Full tuple coverage is the substantive assumption.

4. Candidate generation, checking and feedback

The session holds the fixed request, an incumbent, replayable witnesses, bounded history, a monotonic resource ledger and an attempt index. The initial incumbent is `OriginalAdmitted(P0)`. This is a valid fallback and is reflexively equivalent to itself; it has no transform receipt unless such a check was actually performed. A replacement incumbent instead contains owned immutable candidate bytes and a genuine private checker result. Untrusted receipt files cannot directly construct that result.

The proposer may be neural, symbolic, hand-written or adversarial. The initial interface accepts complete candidate artifacts. A later optional interface accepts bounded strategy data selecting fixed rewrite IDs, iteration counts and supported extractors through an already-qualified interpreter. Neither interface runs model-supplied host code, imports, callbacks or interpreter implementations. Search and extraction remain in the shell, outside the pure acceptance path.

```
admit the original and freeze request R
incumbent := OriginalAdmitted(P0)
for each reservable attempt within R's fixed limits:
    reserve attempt and stage allowances before work
```

```

propose, and optionally run bounded symbolic search
decode the actual emitted candidate Q
report := transform(P0, Q, R)
retain report, costs and unresolved reservations
replay any difference before labeling it factual feedback
select Q only if its genuine equivalence result and cost guards pass
return incumbent, receipt status, stop reason and accounting

```

The checker returns Equivalent, Different, Refused or Inconclusive. Equivalent means full-domain checking completed under the fixed semantics. Different contains a tuple and both exact typed observations. Refused means an admission condition failed. Inconclusive covers insufficient work, unexpected execution failure and unavailable execution. Equivalent candidates may still be reported as equivalent without improvement. These outcomes must not be collapsed into a single success flag controlled by the proposer.

A semantic witness binds the request, original, candidate, ordinal, tuple and both observations. Witness replay recomputes its contents before feedback calls it factual. A model may supply an explanation, but that explanation is advisory. Incomplete checking supplies a stop reason, not a fabricated counterexample. All accepted candidates are compared directly with P0. Prompt history may be bounded or omit older advisory material without weakening the acceptance test.

Here independence means that acceptance does not trust search verdicts, extraction optimality or model assertions. Qualification must examine the actual decoder, evaluator, enumerator and receipt code, using an independently written semantic oracle and meaningful mutations. Running the same mistaken semantics twice, or asking a second model to agree, does not establish soundness. The remaining trusted code and assumptions must be named.

5. Conditional preservation, selection and termination

The following statements are paper-level arguments for the proposed design. They are not newly machine-checked Lean or Verus results. Assume correct admission and correspondence between canonical bytes and execution (A1); complete bijective enumeration and exact observations (A2); a checker that constructs equivalence only after complete successful comparison (A3); immutable request/artifact custody and sound receipt binding/replay (A4); and correct actual cost measurement and fixed selection logic (A5).

Proposition 1 — Incumbent preservation. Under A1–A5, every reachable incumbent I is `equivalent_R` to $P0$. Each replacement has a genuine result for the unchanged request. The claim holds for any adaptive proposer.

Proof sketch. At initialization, $I=P0$ and equivalence follows by reflexivity on the admitted domain. This mathematical fact does not create a receipt. For an induction step, failed proposal, refusal, incomplete checking, semantic difference or nonselection leaves I unchanged. A selected replacement comes from a completed checker result against $P0$. A1–A4 therefore establish the relation for the actual retained bytes. No assumption about the proposer’s probability of success occurs in the argument. Immutable custody prevents a later mutation from substituting different bytes for that established subject.

Measure $C(P)=(N(P),B(P))$, where N counts every stored instruction once and B is the actual canonical artifact length. A candidate Q is selected only if:

```

N(Q) <= N(P0), B(Q) <= B(P0), and at least one is strict;
C(Q) <lex C(I).

```

Proposition 2 — The exact cost guarantee. Every incumbent is componentwise no more costly than the original, and selected incumbent costs strictly descend lexicographically. There are finitely many selections.

Proof sketch. The component bounds hold initially by equality and for each replacement by its guard. The second guard gives lexicographic descent at every update. Costs are nonnegative integers in the finite box determined by the original, so the number of updates is at most $(N(P_0)+1)(B(P_0)+1)-1$. This is a loose finiteness bound, not a performance estimate. Componentwise descent between successive incumbents does not follow: original (12,300), first incumbent (10,200) and next incumbent (9,250) satisfy the rule. Ties keep the incumbent, avoiding identity churn with no objective gain.

Extraction cost and equivalence are independent. A complete feasible candidate from a search of unknown optimality can be checked and selected without proving a global minimum. Its search status must remain visible. Partial or undecodable output cannot be accepted. This prospective product rule does not change the historical study’s positive-optimal-status extraction gate.

Proposition 3 — Bounded attempts. If an attempt is consumed before every proposal, all stage work is bounded, and the supervisor eventually enforces deadlines and cleanup, only finitely many attempts occur and the session ends without promoting an unchecked artifact.

Proof sketch. Each iteration strictly consumes the finite attempt allowance, including repeated invalid or empty proposals. A bounded stage cannot hold the loop indefinitely under the supervision assumption. Completed failures retain the incumbent by Proposition 1; incomplete stages cannot produce a private success result. Finite cost descent alone would not prove this: a model could otherwise fail forever without changing the incumbent. Deadline enforcement does not assert hard-real-time scheduling or guaranteed cancellation of a remote provider’s computation.

These arguments establish neither search completeness nor convergence to an optimum. They do not prove that P_0 meets human intent, that the finite domain captures an application, or that the implementation satisfies A1–A5. Those are distinct specification and executable-refinement obligations.

6. Resource accounting, provenance and restart

The design starts with conservative limits to be measured and qualified, not benchmark-derived optimal settings. Each artifact is at most 64 KiB. A session permits eight attempts and at most eight candidate checks, four model calls, 4,096 input and 2,048 output tokens per call, and 24,576 reserved tokens in total. A check permits at most 1,000,000 deterministic work units; the session permits 8,000,000 including replay. These ceilings intersect with the smaller limits in the reviewed request.

A search worker has a five-second external deadline and a 512 MiB memory limit including descendants, with one solver thread and bounded I/O. A standalone checker worker has a two-second deadline and 64 MiB limit. The whole session has a 20-second deadline across calls, search and checking, so not every individually permitted stage is guaranteed to fit. Cleanup latency is reported separately. On a supported Linux host, an owned cgroup with memory.max, no swap and descendant termination can enforce the process limits. Other hosts need qualified equivalent controls. Missing enforcement means unavailable search. A between-batch e-graph node threshold or a solver’s solve timeout is not a substitute for these bounds.

Deterministic core work is charged before work, including parsing, allocation, node attempts, comparisons and receipt construction. Refusal preserves measured usage and the attempted charge.

The shell separately reserves attempts, tokens and the maximum authorized monetary charge before dispatch. Hosted models are disabled by default with a zero monetary allowance. Enabling a provider requires an explicit disclosure and billing policy; standing operator configuration may authorize ordinary requests within finite aggregate budgets. No new approval dialog is needed for work already covered by that authorization. A provider whose spending limit cannot be enforced is unavailable under that policy.

A timeout with unknown remote completion retains its reservation and reports incomplete accounting. It is not a free failed call. Local caps do not constrain a hosted provider's internal memory. Credentials never enter transcripts or receipts; private source is disclosed only within the operator's configured scope. Provider/model names, prompts and response digests, decoding settings, seeds and reported billing remain provenance. They cannot supply semantic authority. Checker-derived actual costs and deterministic usage, in contrast, are checked receipt fields.

Persistence contains immutable requests, actual artifacts, receipts and a shell-owned reservation ledger. Resume matches the exact request and source identity, re-admits P0 and replays any incumbent before use. Witnesses reused as facts are replayed too. Replay consumes remaining allowance. Failed resume returns no trusted incumbent; stale saved bytes do not inherit their old label. Unknown pending reservations remain charged unless the approved policy can reconcile them. A rollback-suspected ledger fails closed; a self-hash alone does not prevent rollback. Host integrity and durable accounting are explicit assumptions. Automatic new sessions must not reset exhausted budgets.

7. Receipts and the application boundary

A deterministic equivalence receipt binds the exact request and both actual artifacts, lengths and codec; complete ordered domains and ABI; execution, observation, error and enumeration versions; checker source closure and build; expected and visited rows, terminal state and trace commitment; actual costs; and work policy and usage. Replay re-admits the actual bytes, reruns the full check and compares deterministic fields. A stale manifest invalidates dependent evidence. Receipt parsing alone cannot construct a checked replacement.

The loop fits the existing synthesis MCP and skill integration. A narrow request, candidate-feedback and replay interface is proposed; no new installed server or tool capability is claimed here. The shell may manage sessions and providers, but the library owns checked result construction. An equivalence receipt creates neither Authority nor Evaluation nor Publication capability. The existing authority route remains the only application publication path.

Functional equality is smaller than application equivalence. Applications can observe decision class, rejection reason and precedence, successor state, canonical patch, ordered effects and outbox contents, destinations, law and genesis frames, protected traces, resource charges and refusal. Source and contract identity, replay and publication binding also matter. Two programs with equal Boolean outputs can produce different whole decisions if removing instructions changes a budget refusal or a resource-observing law.

Therefore resource refinement is a separate relation requiring a reviewed policy over inputs and supported budgets. It must explicitly decide whether an old refusal may become success and preserve charge-before-work accounting. Different source identities likewise require a reviewed old/new mapping and evidence dependencies; they cannot be erased to make publication bytes equal. Historical replay retains its original artifact and policy. Optimization does not reinterpret pending effects or past records.

A later checked-i64 profile must preserve exact checked arithmetic and eager errors, including overflowing unused nodes and unselected branches. Its full domain must fit a declared bound; the proposed ceiling is 65,536 tuples, not the entire i64 range. Full applications need complete raw-domain coverage or a proved refinement bridge from it. A small set of Boolean guard combinations cannot by itself qualify a large application state space.

8. The completed symbolic case study

The empirical input is *Checked Semantic Equality Saturation for Finite Boolean Programs: An Exploratory ZenoFCIS Case Study*, dated October 3, 2026 [7]. Its immutable published revision is `a69ed8db594d95279a46bff0f65185ef67d51f98`. The subtree contains source, corpus, protocol, original and repeat receipts, failed runs, analyzer, Lean model and paper. It measures symbolic optimization; there is no online neural proposer.

The fixed synthetic corpus has 100 cases in six families, with three to six Boolean inputs and ordered, sometimes repeated outputs. Four variants per case produce 400 artifacts: original, strong local simplification, rewrite-only equality saturation and semantic equality saturation. Exact Boolean signatures are finite truth tables, with a special full-width mask for 64 rows. Negation is masked to the admitted table width. The study lowers extracted expressions to a shared multi-output DAG and counts stored instructions once, rather than summing independent output trees.

Variant	Aggregate emitted instructions	Interpretation
Original	1,506	Unoptimized emitted programs
Local simplification	895	Strong local baseline with sharing
Rewrite-only e-graph	854	Symbolic rewrite search
Semantic e-graph	816	Adds complete finite signatures
Protected selection	812	Study-specific selection between local and semantic

Semantic search improves 18 cases, ties 78 and regresses on four relative to local simplification; improvements occur in three families. Relative to rewrite-only search it improves 12 cases and ties 88. The protected selection chooses semantic results for 18 cases and local results for 82. Its aggregate 812 uses the study’s own node-count and extraction-status conditions. It does not evaluate the new pair of node count and canonical byte length, request-bound session logic or production receipt design. No byte-size gain or neural gain has been established by these numbers.

The experiment’s acceptance chain checks actual lowered programs as well as expressions. Reported Rust evaluation and independent Python expression and serialized-DAG interpreters compare complete ordered behavior. Each successful run records 12,544 admitted comparisons and 8,176 invalid-input probes. Boolean admission rejects arithmetic even in unused or unselected nodes. The separate arithmetic controls illustrate why an integer extension must preserve eager errors; replaying their stored flags is not a fresh arithmetic execution.

Two earlier configurations remain failures. Six iterations, a 5,000-node threshold and a two-second extraction setting produced 12 failures, including decoding failures and rejected nonoptimal extraction. Increasing extraction time to 30 seconds left one failure. The successful configuration used three iterations, a 1,000-node threshold and a 30-second extraction setting, selected after those outcomes. It is exploratory, not a preregistered confirmatory configuration. The maximum recorded

e-graph size was 1,351, demonstrating why that threshold is not a hard memory cap. A solver time setting likewise does not bound model construction, reconstruction or total execution.

The reported Lean development proves abstract complete-signature and guarded acceptance properties. It does not prove correspondence with the Rust decoder, evaluator, enumerator, solver or application authority. The published proof receipts and declared standard Lean axioms must retain that boundary. The published egg fork and solver pins aid reproduction, but the vendor subtree is absent; its documented setup, exact native dependencies and licenses still require qualification before product adoption.

9. Current evidence and what was not rerun

A bounded October 4 review checked all 54 manifest-listed artifact hashes. Fresh Python analysis of both successful stored runs reproduced all published non-timing analysis fields. Python-version differences changed only timing-sum rounding, by at most 3.64e-12 milliseconds. Repeat program, behavior, node-cost, status and error fields agree. All 13 analyzer self-test mutations were rejected, and both failed configurations were correctly rejected [8]. A separate source check verified program/signature digests and the documented proof-file bindings [9]. Agreement between two stored runs is observed repeatability, not a proof that every future execution is deterministic.

Evidence layer	Status in this draft	Supported claim
Original Rust optimizer and Lean runs	Reported in published source/receipts	Historical bounded symbolic results and abstract theorem boundary
Current Python analysis/hash replay	Executed on stored artifacts	Their recorded Boolean results, bindings and analyzer controls replay
Adaptive-loop arguments in Sections 3–7	Proposed paper proofs	Conditional claims under explicitly stated assumptions
New decoder/checker/session implementation	Not implemented or qualified here	No new executable soundness claim
Neural, target and whole-application studies	Not performed	No neural gain, target equivalence or application authority claim

The current review did not rebuild the optimizer, run native Rust tests or recompile Lean. Analyzer replay is not execution of those tools. The analyzer also does not independently validate every provenance/checksum label or execute the arithmetic controls. These limitations do not invalidate the replayed Boolean numerical claims; they prevent historical analysis from becoming a production acceptance mechanism. New strict receipts must validate every required binding against the actual source and artifacts.

10. A falsifiable neural evaluation plan

Any claim that the neural component improves discovery requires a separately authorized experiment. Before tuning, preregister independently accepted held-out task contracts, task and repeat counts, provider/model versions, decoding parameters, seeds, total budgets, stopping rules and missing-data treatment. The old 100-case corpus is calibration material only. Accepting a task

means its contract was reviewed independently of whichever optimizer will attempt it; it does not mean all human intentions have been formally captured.

The minimum comparison has six arms: local-only simplification; fixed rewrite-only e-graph search; semantic e-graph search; model-only whole-program proposals with the same checker; a hybrid without checker feedback in its prompts; and the hybrid with typed feedback. The no-feedback hybrid still uses the acceptance checker and safety gates. Thus the ablation removes discovery feedback, not correctness checks. A later optional comparison can examine whole candidates versus bounded strategy proposals.

Use the same domain, observation, original-cost guards, checker and total resource ceilings for every arm. An arm need not spend its unused model allowance, but report actual spend. Equal ceilings alone do not show economic superiority; report quality alongside time, tokens and money, and preregister any cost-matched secondary comparison. Count every assigned task and run, including invalid proposals, refusals, timeouts and no-improvement outcomes. Do not increase a failed arm’s budget after looking at results and call that the original comparison.

Primary outcomes are checked actual node and byte deltas and the fraction of tasks obtaining a checked improvement. Report total calls, tokens, monetary charges and unresolved reservations, wall time, time to first complete pass and time to first improvement. Runs without those events are censored at their actual stopping limits, not omitted. Report semantic failures, admission failures and incomplete checks separately. Record human review effort and attempts to alter the frozen domain, observations or policy.

Repeat stochastic trials, retain every result, and report paired uncertainty over tasks using a preregistered aggregation and interval method. Multiple retries on one task are not independent task samples. Provider drift and unobservable hosted state limit exact reproduction even when request digests and seeds are logged. No best-of-many-only table should stand in for the full outcome distribution.

Checker qualification must also include known differences and deliberately planted checker defects, such as omitted rows, ignored output positions, weakened receipt bindings or success on incomplete work. Measure false acceptance or mutant detection with explicit denominators on an isolated test path. Zero observed false acceptance is evidence about that injected population, not a universal soundness proof. No fault-injected checker participates in an application authority route. A null or negative neural result is an acceptable outcome and should remain in the record.

11. Implementation obligations and limitations

The smallest implementation begins with supplied-candidate Boolean transform, not a model service. Qualify canonical decoding, total Boolean semantics, enumeration coverage, complete comparison, immutable result custody and charge-before-work. Then add strict source-bound receipts and replay, resource and resume tests, bounded local optimization, and a fake-provider adapter in the existing synthesis integration. A real provider is optional and follows those gates. Exact source/body coverage, formal refinement, native comparisons, meaningful mutations and independent review must concern the version enabled for use, not only the historical study.

The proof obligations are intentionally narrower than verifying an optimizer. A poor proposer may still produce a checkable candidate. Conversely, a strong optimizer cannot compensate for an unsound checker or inadequate observation profile. Complete finite checking scales exponentially with Boolean arity and with products of integer ranges. Increasing a bound is a contract and cost decision, not a free extension of the evidence. If application coverage or target correspondence is

impractical, those modes remain unavailable.

This design assumes a trustworthy execution host for qualified checker binaries, artifact custody and durable accounting. It does not solve a malicious kernel, provider billing outside an enforceable contract, or side channels omitted from the observation profile. Hard resource controls constrain local workers; they do not prove real-time responsiveness. Dependency/fork and native solver license review, fresh optimizer/proof execution and all new neural evaluation remain unperformed acceptance work. None should be marked complete because the documentation or historical artifact hashes are consistent.

12. Conclusion

A neurosymbolic software factory can assign neural systems an ambitious discovery role while keeping acceptance under explicit symbolic contracts. For the bounded ZenoFCIS transformation lane, that means an immutable original request, complete checking of actual artifacts, guarded incumbent selection and honest handling of failure, accounting and replay. The symbolic study provides limited empirical support for semantic search. Whether a neural policy improves this process remains a testable hypothesis. The proposed contract makes that hypothesis evaluable without promoting search success into application authority or correctness of human intent.

AI assistance and artifact statement

This revised draft and its design/specification packet were prepared with assistance from OpenAI’s gpt-6-astra at maximum reasoning effort, with agent coordination and document review. Dana Edwards is the named author. AI assistance is not an additional human coauthor and is not evidence of independent implementation verification. No online model optimization experiment, new optimizer execution or new Lean/Verus proof execution was performed for this draft. The original published study remains unchanged; the current replay receipts identify their narrower executed scope.

References

- [1] Ross Tate, Michael Stepp, Zachary Tatlock and Sorin Lerner. *Equality Saturation: A New Approach to Optimization*. POPL, 2009. Author publication page.
- [2] Max Willsey et al. *egg: Fast and Extensible Equality Saturation*. Proceedings of the ACM on Programming Languages, POPL, 2021. Primary preprint.
- [3] Armando Solar-Lezama et al. *Combinatorial Sketching for Finite Programs*. ASPLOS, 2006. Author PDF.
- [4] Zhenyang Xu, Hongxu Xu, Yongqiang Tian, Xintong Zhou and Chengnian Sun. *LPO: Discovering Missed Peephole Optimizations with Large Language Models*. ASPLOS, 2026. DOI: 10.1145/3779212.3790184. Author PDF.
- [5] Chenyun Yin, Youwei Xiao, Yuze Luo, Yuyang Zou and Yun Liang. *LLM-Guided Strategy Synthesis for Scalable Equality Saturation*. arXiv:2604.17364, version 1, April 19, 2026. Preprint. Primary record.
- [6] Sijie Kong et al. *Improving Equality Saturation for EDA via Semantic E-Graphs*. PLDI, 2026. Author PDF.

- [7] Dana Edwards. *Checked Semantic Equality Saturation for Finite Boolean Programs: An Exploratory ZenoFCIS Case Study*. Draft, October 3, 2026. Published experiment revision a69ed8db594d95279a46bff0f65185ef67d51f98. Immutable artifacts.
- [8] ZenoFCIS review artifacts. *Stored-result Python replay receipt*, October 4, 2026. Receipt.
- [9] ZenoFCIS review artifacts. *Bounded source and digest check*, October 4, 2026. Receipt.